Invention Disclosure Form (Draft)

Prediction-aware, rank-ordered, work-conserving task dispatch with a log-fitted generic runtime model

Lakshay S (26MAI0064), Arkkadu Bharath Reddy (26MAI0058) | Version of 3 November 2026

Template note (Lab 9). The institution's IDF-B template was not available in this work; the sections below follow the usual IDF structure (administrative data, field, background, problem, summary, embodiments, prior art, advantages, inventor contributions) so that the text can be moved into the official form.

Status of this draft. This is a working draft for discussion with the institution's IPR cell or a patent professional. It states what the project built and measured. It does not claim that the described elements are novel or non-obvious; the prior-art section lists documents that may anticipate or render obvious parts of them. All fields in square brackets are to be completed by the authors.

1. Administrative data

Table 1. Form fields

Field	Entry
Title of the invention (working)	Dispatch of dependent tasks on heterogeneous workers using a runtime model fitted from execution logs
Inventors	Lakshay S (26MAI0064); Arkkadu Bharath Reddy (26MAI0058); guide Dr. Yokesh Babu S, Associate Professor Grade 1 (whether the guide is a co-inventor depends on his contribution to the inventive concept and is for the IPR cell to decide)
Guide / supervisor	Dr. Yokesh Babu S, Associate Professor Grade 1
Institution, department	[to be filled]
Funding / sponsor / third-party material	[none known; third-party: Montage workflow software, DSS2 images, open-source Python libraries]
Date of conception; dates of first reduction to practice	[authors to confirm from the git history; the repository has 71 commits on branch revival/resource-state-aware]
Any public disclosure so far?	[none known to the assistant; the authors must confirm: talks, reports submitted to the course, public repositories, preprints]
Intended publications	see 07_Publication_and_IPR_Plan; none before a filing decision

2. Technical field and background

The invention relates to scheduling the tasks of a dependency graph on a heterogeneous computing system that includes at least two kinds of processor cores (for example performance and efficiency cores), optionally a graphics processing unit and optionally a remote worker reached over a network.

Background. In eligibility-gated dispatch (the idea mapped from US 11,561,826 B1 [1]), a ready task is handed to the first idle worker for which it is eligible, using no estimate of execution time. Where the cost of tasks differs in a way that the scheduler cannot see (for example, a task parameter that does not appear in the input size), such dispatch can start expensive tasks late or on slow workers. List schedulers such as HEFT and PEFT ([2], [3]) order tasks using rank values from a given cost matrix. Learned schedulers ([4], [5]) train policies in simulation. IBM [6] describes a learning agent that assigns a rank to tasks of a DAG on a heterogeneous system on a chip.

Problem. To obtain an ordering that accounts for hidden task cost without (a) hard-coding which task parameters matter, (b) training a neural policy in simulation, or (c) relying on a person to sort the tasks, while keeping the scheduler's own decision cost low and measured.

3. Summary of the implemented system

The system comprises a coordinator, a set of worker processes pinned to processor cores (and optionally a GPU worker and a remote worker), a dependency graph of tasks with declared eligible worker types, and a scheduler policy with two hooks: ordering the ready tasks and selecting a worker.

1. **Model fitting.** From logs of earlier executions (each record: task stage, worker type, numeric task parameters, input size, measured kernel time, and the occupancy context), fit for each (stage, worker type) pair a regularised linear regression whose features are those numeric parameters that vary in the logs and their pairwise products. No stage name or parameter name appears in the code that selects features.
2. **Guard.** If a stage or worker type was not present in the fitting data, the model raises an error instead of returning an estimate; a profile identifier recorded at fitting time is compared with the profile in use and a mismatch is refused.
3. **Ranking.** For the ready tasks, compute the predicted upward rank: the predicted time of the task plus the longest path of predicted times (and predicted transfer times, where a remote worker is a candidate) to an exit task.
4. **Dispatch.** Order the ready tasks by decreasing predicted rank. For each task in that order, consider only workers that are idle at that moment and for which the task is eligible; choose the one with the smallest predicted execution time (plus predicted transfer time); never place a task in the queue of a busy worker.
5. **Accounting.** Record for each dispatch the elapsed decision time and, separately, the total time the coordinator spent in policy calls; flag the case in which an idle worker that no ready task can use prevents the dispatch loop from exiting early.
6. **Refit.** For a new pool layout, rerun the fitting command on logs gathered under that layout.

An optional pre-deployment gate replays recorded logs with an oracle estimator and enables predictive scheduling only if the achievable makespan reduction exceeds a configured threshold.

4. Embodiments and evidence

The embodiment was implemented in Python (about 12,600 lines runtime, 521 automated tests) and executed on an Intel Core i5-13500HX laptop with 6 performance cores, 8 efficiency cores and an

NVIDIA RTX 3050 GPU, and, in emulation, with a remote worker behind a loopback socket. Measured results (real executions, paired makespan changes against B4):

Table 2. Measured results supporting the description

Workload	Result	Note
64 image chains, 16 heavy (60x), 6P+7E, 12 unseen chain orders	−19.32% [−20.0, −18.6]	faster in 12 of 12 orders; exploratory
same, 2P+7E, 8 unseen orders (model refitted by one command)	−20.45% [−21.2, −19.8]	faster in 8 of 8
4 or 8 heavy chains of 64	−27.65%; −26.76%	gain largest with few heavy tasks
40 heavy chains of 64	−3.81% [−5.1, −1.9]	gain shrinks
cost ratio 7.5 / 20 / 60	−1.3% / −2.9% / about −20%	small below a ratio of about 20
uniform batch (nothing hidden)	+0.45%	equivalent within 2%
Montage mosaic	no gain; B4 +47.9% vs static P-only	no headroom
informed hand rule vs proposed	+0.66%, +0.27% (hand rule)	level: the dispatcher does not beat a person who knows the parameter
HEFT with the same model vs proposed	+1.10% [−0.53, +2.81]	level
emulated remote worker, level 30	EFT +3.78% slower with remote	no offloading gain
final benchmark, primary workload, CPU (24 blocks, Holm)	PRED vs B4 −17.8% [−18.8, −16.7]; vs static HEFT −5.2% [−5.9, −4.6]	confirmatory; end-to-end makespan
final benchmark, strict GPU set	PRED vs B4 −21.2% [−22.6, −19.7]; vs static HEFT −12.2% [−13.4, −11.1]	GPU used only where a speed-up was measured
final benchmark, PRED vs hand rule B4_param_first	−0.4% [−0.9, +0.1] (CPU)	within 1% in every configuration (exploratory)

5. Prior art located (not exhaustive)

Table 3. Documents to be considered by the examiner or professional

Document	Relevance
[1]	task graph, kernel queues, in-degree gating, thread adjustment; no prediction
[6]	learning agent assigns rank to DAG tasks; rank-ordered dispatch on heterogeneous SoC
[7]	learned DAG policy, task execution estimates, drop decisions
[8]	CPU+GPU work-split by online profiling
[9]	task duration prediction for heterogeneous cores (lapsed)
[10], [11]	static graph-to-processor mapping; ML workload placement
[2], [3]	list scheduling by upward rank and optimistic cost table
[12], [13]	history-based performance models; runtime prediction for workflows
[4]	learned task selection with duplication-based placement

The search was limited to Google Patents pages and the project's literature set; a professional search is required.

6. Possible advantages (to be argued by the authors, not asserted)

- No hard-coded stage or parameter knowledge; the model discovers varying parameters from logs.
- Low decision cost (15 ms of decision time per run of 257 tasks) and measured overhead.
- One-command refit for a new pool layout, demonstrated for two layouts.
- Reported gain where task cost varies (17% to 40% against blind dispatch; 2% to 12% against a static plan from the same model); no harm where it does not.
- Limits that a reviewer will note: equal to an informed hand rule; emulated remote; one machine; designed workload.

7. Inventor contributions and signatures

Table 4. To be completed

Item	Entry
Conception of the dispatcher and model (who, when)	[authors]
Implementation and experiments (who, when)	[authors]; assistant-generated code and analysis were used in the project: record the extent of AI assistance as the institution's policy requires
Signatures and dates	[authors, guide, witness as required]

References

[1] S. Nagpal, A. Karumannil, V. Jain, and A. K. Patil (Xilinx, Inc.), “Scheduling processing of machine learning tasks on heterogeneous compute circuits,” U.S. Patent 11,561,826 B1, Jan. 24, 2023 (filed Nov. 12, 2020).

[2] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[3] H. Arabnejad and J. G. Barbosa, “List scheduling algorithm for heterogeneous systems by an optimistic cost table,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014, doi: 10.1109/TPDS.2013.57.

[4] J. Luo, Y. Zhou, X. Li, M. Yuan, J. Zeng, and J. Yao, “Learning to optimize DAG scheduling in heterogeneous environment,” in *Proc. IEEE Int. Conf. Mobile Data Manage. (MDM)*, 2022, doi: 10.1109/MDM55031.2022.00040 (arXiv:2103.06980).

[5] Q. Tan, W. Chen, and D. Liu, “Task scheduling for heterogeneous multi-core processors based on deep reinforcement learning,” *Int. J. Intell. Syst.*, vol. 2025, Art. no. 7562400, 2025, doi: 10.1155/int/7562400.

[6] A. Vega *et al.* (International Business Machines Corp.), “Heterogeneous system on a chip scheduler with learning agent,” U.S. Patent 11,740,933 B2, Aug. 29, 2023.

[7] A. Amarnath *et al.* (International Business Machines Corp.), “Learning agent based application scheduling,” U.S. Patent 11,966,776 B2, Apr. 23, 2024.

[8] R. Barik, T. Shpeisman, B. T. Lewis, and R. Kaleem (Intel Corp.), “Adaptive scheduling for task assignment among heterogeneous processor cores,” U.S. Patent 10,186,007 B2, Jan. 22, 2019.

[9] R. Kim and M.-S. Kim (Samsung Electronics Co., Ltd.), “Method and apparatus for performing task scheduling in terminal,” U.S. Patent 10,162,671 B2, Dec. 25, 2018.

[10] B. P. Englert, E. B. Harris, N. G. Crane, and B. J. Corey (Apple Inc.), “Scheduling heterogeneous execution on heterogeneous hardware,” U.S. Patent 11,775,811 B2, Oct. 3, 2023.

[11] J. Wang and H. Huang (Google LLC), “Task scheduling for machine-learning workloads,” U.S. Patent 11,544,113 B2, Jan. 3, 2023.

[12] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, “StarPU: A unified platform for task scheduling on heterogeneous multicore architectures,” *Concurrency Comput.: Pract. Exper.*, 2011 (journal version; see also Euro-Par 2009, LNCS 5704, pp. 863–874).

[13] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, “Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures,” *Future Gener. Comput. Syst.*, vol. 150, pp. 171–185, 2024, doi: 10.1016/j.future.2023.08.022.